

云基础设施 · AI Infra

缓存、传输与调度：把系统题回答成一套可验证的代价模型

KV Cache

P/D 解耦

RDMA / Collective

系统设计

岗位能力画像

岗位画像：不仅考缓存知识，更考是否能把数据位置、网络、队列和失败恢复放进同一个判断框架。

高频深挖：题目背后的判断

远端缓存何时值得复用？

命中不等于更快；先比较复用收益与查表、传输、排队、迁移成本。

$reuse\ gain > lookup + transfer + queue + migration$

P/D 分离如何避免只是在搬数据？

说明谁生产 KV、谁消费 KV、什么时候传，以及失败后谁负责回收。

资源异构与调度边界，而不是一句“降低延迟”。

Ring 和 Tree 怎么选？

把消息大小、拓扑、并发和计算 overlap 说清，拒绝背固定结论。

大消息看带宽，小消息看延迟，先测再选。

系统设计从哪里开始？

按请求-调度-缓存-worker-回写拆层，并为每层给可观测指标。

每层都有输入、状态、失败路径和指标。

项目叙事的通用深挖链

问题与约束

证据与机制

方案与取舍

验证与边界

答题提醒

不要从‘用了什么库/指令’开始；先固定工作负载、指标和约束，再说明数据流/资源/依赖，最后给出正确性、性能和反例边界。

把一句结论变成可复验的事实

现象

固定 workload / 阶段 / 指标，明确‘慢’或‘错’在哪里。

证据

用 trace、counter、对照或最小复现排除相邻假设。

机制

说明资源、数据流或依赖如何导致这个现象。

验收

数值、边界、重复性和 E2E 共同成立后再下结论。

一题答透：分布式 KV Cache：先算账，再设计

问题 “已有前缀命中时，怎样判断远端复用而不是本地重算？”

1. 固定目标

区分在线的 TTFT/尾延迟与离线的总吞吐；两类负载不能共用一句‘更快’。

2. 写代价模型

复用节省的 prefill 计算，必须大于 block 查找、链路传输、队列等待和一致性维护成本。

3. 设计状态

用 block table 描述 ownership、ready、lease、evict；超时和重试必须有幂等 transfer id。

4. 回到验收

同时看 cache hit、传输带宽、queueing、TTFT/TPOT、失败率与显存碎片。

把推理链画出来



常见误区

× 只说‘命中缓存就快’，没有写传输和排队成本。

× 把 P/D 当成万能加速，忽略低负载下的额外控制面开销。

× 没有缓存失效、超时、重试和可观测性设计。

术语速懂 | 面试怎么答

术语速懂

P/D 分离

Prefill 偏算力、Decode 偏长 KV 与尾延迟；拆分后必须定义 KV 的传输与回收。

Lease

短期持有权：消费者在使用 block 前取得 lease，超时后可安全回收或重试。

面试中怎么答

- ① 先用 TTFT、TPOT 和负载比例判断拆分是否值得。
- ② 再说明 owner / ready / lease、幂等传输和超时回收。
- ③ 最后同时验证队列、带宽、尾延迟、命中率和失败恢复。

白板 / 手撕：白板题：区间合并 + 缓存块表

题面 给一组可能重叠的区间，合并后输出；再讨论如何把连续 token 切为固定块并维护块表。

C++17 · interval merge

核心实现

```
01 struct Seg { int l, r; }; // half-open [l, r)
02 vector<Seg> merge(vector<Seg> a) {
03     sort(a.begin(), a.end(),
04         [](const Seg& x, const Seg& y) {
05             return x.l < y.l; });
06     vector<Seg> out;
07     for (const auto& s : a)
08         if (out.empty() || s.l > out.back().r)
09             out.push_back(s);
10         else out.back().r = max(out.back().r, s.r);
11     return out;
12 }
```

写代码时怎么讲

1. 先按左端点排序，维护当前 $[l, r]$ 。
2. $\text{next.l} \leq r$ 时扩展 r ，否则输出当前区间并开新区间。
3. 缓存块表再补 key、owner、ready、lease 和 eviction 字段。
4. 说明复杂度 $O(n \log n)$ 与空输入、边界相接、重复区间测试。

实现重点

- 复杂度：排序主导 $O(n \log n)$ 。
- 边界：空输入、相接/包含、重复区间。
- 延展：块表另维护 id、ready、refcount、lease。

边界、测试与追问

最少要说明的验证

- 复杂度：排序主导 $O(n \log n)$ 。
- 边界：空输入、相接/包含、重复区间。
- 延展：块表另维护 id、ready、refcount、lease。

面试继续追问

- 大量定时回调为什么优先 min-heap？
- block key 如何避免 token 顺序交换导致错误命中？
- 传输超时后如何保证重试不重复消费？